

Pemrograman Berorientasi Objek



PHP and JavaScript 00

Adapted from:

<https://www.w3schools.com/>

PHP & OOP

- ▶ From PHP5, you can also write PHP code in an object-oriented style.
- ▶ Object-Oriented programming is faster and easier to execute.

Case

- ▶ Let's assume we have a class named Fruit. A Fruit can have properties like name, color, weight, etc. We can define variables like \$name, \$color, and \$weight to hold the values of these properties.
- ▶ When the individual objects (apple, banana, etc.) are created, they inherit all the properties and behaviors from the class, but each object will have different values for the properties.

Creating the Fruit Class

```
<?php
class Fruit {
    // Properties
    public $name;
    public $color;

    // Methods
    function set_name($name) {
        $this->name = $name;
    }
    function get_name() {
        return $this->name;
    }
}
?>
```

The \$this Keyword

- ▶ The \$this keyword refers to the current object, and is only available inside methods. Look at the following example:

```
<?php
class Fruit {
    public $name;
    function set_name($name) {
        $this->name = $name;
    }
}
$apple = new Fruit();
$apple->set_name("Apple");
echo $apple->name;
?>
```

Creating objects

```
<?php
```

```
$apple = new Fruit();
```

```
$banana = new Fruit();
```

```
$apple->set_name('Apple');
```

```
$banana->set_name('Banana');
```

```
echo $apple->get_name();
```

```
echo "<br>";
```

```
echo $banana->get_name();
```

```
?>
```

Access modifiers in PHP

- ▶ Properties and methods can have access modifiers which control where they can be accessed.
- There are three access modifiers:
 - **public** - the property or method can be accessed from everywhere. This is default
 - **protected** - the property or method can be accessed within the class and by classes derived from that class
 - **private** - the property or method can ONLY be accessed within the class

Which lines raise error(s)?

```
<?php
```

```
class Fruit {  
    public $name;  
    protected $color;  
    private $weight;  
}
```

```
$mango = new Fruit();  
$mango->name = 'Mango';  
$mango->color = 'Yellow';  
$mango->weight = '300';
```

ERRORS

```
?>
```


Access modifier in methods

```
<?php
class Fruit {
    ...
    function set_name($n) { // a public function (default)
        $this->name = $n;
    }
    protected function set_color($n) { // a protected function
        $this->color = $n;
    }
    private function set_weight($n) { // a private function
        $this->weight = $n;
    }
}
$mango = new Fruit();
$mango->set_name('Mango'); // OK
$mango->set_color('Yellow'); // ERROR
$mango->set_weight('300'); // ERROR
?>
```

Constructors

- ▶ `__construct()` is a function that is automatically called when you create an object from a class

```
<?php
```

```
class Fruit {  
    public $name;  
    public $color;
```

```
    function __construct($name) {  
        $this->name = $name;  
    }
```

```
    function get_name() {  
        return $this->name;  
    }
```

```
}
```

```
?>
```

Destructor

- `__destruct()` is a function that is automatically called at the end of the script

```
<?php
class Fruit {
    public $name;
    public $color;

    function __construct($name) {
        $this->name = $name;
    }
    function __destruct() {
        echo "The fruit is {$this->name}.";
    }
}

$apple = new Fruit("Apple");
?>
```

Inheritance in PHP

- ▶ Similar to Java, Inheritance = When a class derives from another class.
- ▶ The child class will inherit all the public and protected properties and methods from the parent class.
- ▶ In addition, it can have its own properties and methods.
- ▶ An inherited class is defined by using the **extends** keyword.

PHP inheritance example (1)

```
<?php
```

```
class Fruit {  
    public $name;  
    public $color;  
    public function __construct($name, $color) {  
        $this->name = $name;  
        $this->color = $color;  
    }  
    public function intro() {  
        echo "The fruit is {$this->name} with color is {$this->color}.";  
    }  
}  
?>
```

PHP inheritance example (2)

```
<?php
// Strawberry is inherited from Fruit
class Strawberry extends Fruit {
    public function message() {
        echo "Am I a fruit or a berry? ";
    }
}
$strawberry = new Strawberry("Strawberry", "red");
$strawberry->message();
$strawberry->intro();
?>
```

Accessing “protected” from parent

```
<?php
```

```
class Fruit {  
    public $color;  
    public function __construct($color) {  
        $this->color = $color;  
    }  
    protected function intro() {  
        echo "The fruit color is {$this->color}.";  
    }  
}
```

```
class Strawberry extends Fruit {  
    public function message() {  
        echo "This is the detail about the fruit";  
        $this -> intro();  
    }  
}  
?>
```



This is OK

Overriding Inherited Methods (1)

```
<?php
class Fruit {
    public $name;
    public $color;
    public function __construct($name, $color) {
        $this->name = $name;
        $this->color = $color;
    }
    public function intro() {
        echo "The fruit is {$this->name} and the color is {$this->color}.";
    }
}
?>
```


Overriding Inherited Methods (2)

```
<?php
class Strawberry extends Fruit {
    public $weight;
    public function __construct($name, $color, $weight) {
        $this->name = $name;
        $this->color = $color;
        $this->weight = $weight;
    }
    public function intro() {
        echo "The fruit is {$this->name}, the color is {$this->color}, and
            the weight is {$this->weight} gram.";
    }
}

$strawberry = new Strawberry("Strawberry", "red", 50);
$strawberry->intro();
?>
```

instanceof in PHP

- ▶ You can use the instanceof keyword to check if an object belongs to a specific class:

```
<?php  
$apple = new Fruit();  
echo ($apple instanceof Fruit);  
?>
```

Traits

- ▶ PHP only supports single inheritance: a child class can inherit only from one single parent.
- ▶ What if a class needs to inherit multiple behaviors? OOP traits solve this problem.
- ▶ Traits are used to declare methods that can be used in multiple classes.
- ▶ Traits can have methods and abstract methods that can be used in multiple classes, and the methods can have any access modifier (public, private, or protected).
- ▶ Traits are declared with the trait keyword

Trait example

```
<?php
```

```
trait message1 {  
    public function msg1() {  
        echo "OOP is fun! ";  
    }  
}
```

```
class Welcome {  
    use message1;  
}
```

```
$obj = new Welcome();  
$obj->msg1();  
?>
```

Package (in Java) == Namespaces (in PHP)

- ▶ Namespaces are qualifiers that solve two different problems:
 - They allow for better organization by grouping classes that work together to perform a task
 - They allow the same name to be used for more than one class
- ▶ For example, you may have a set of classes which describe an HTML table, such as Table, Row and Cell while also having another set of classes to describe furniture, such as Table, Chair and Bed. Namespaces can be used to organize the classes into two different groups while also preventing the two classes Table and Table from being mixed up.

Namespace example

```
<?php
namespace Html;
class Table {
    public $title = "";
    public $numRows = 0;
    public function message() {
        echo "<p>Table '{$this->title}' has {$this->numRows}
            rows.</p>";
    }
}
$table = new Table();
$table->title = "My table";
$table->numRows = 5;
?>
```

Can be used with this :

```
<?php
$table1 = new Html\Table()
$table2 = new Furniture\Table();
?>
```

Constants in PHP (1)

- ▶ In PHP, the **final** keyword is being used to prevent inheritance of a class:

```
<?php  
final class MyClass {  
    public $name = "John";  
}
```

```
// This code will throw an error  
class AnotherClass extends MyClass{};  
?>
```

Constants in PHP (2)

- To create class constants, use **const** keyword
- We can access a constant from outside the class by using the class name (or "**self**") followed by operator (::<) then followed by the constant name

```
<?php
class Goodbye {
    const LEAVING_MESSAGE = "Thank you for visiting
                             W3Schools.com!";
}

echo Goodbye::LEAVING_MESSAGE;
?>
```


Static properties in PHP

```
<?php  
class pi {  
    public static $value = 3.14159;  
}  
  
// Get static property  
echo pi::$value;  
?>
```

Static methods in PHP

```
<?php  
class greeting {  
    public static function welcome() {  
        echo "Hello World!";  
    }  
}
```

```
// Call static method  
greeting::welcome();  
?>
```

Abstract class in PHP

```
<?php
abstract class ParentClass {
    abstract protected function prefixName($name);
}

class ChildClass extends ParentClass {
    public function prefixName($name) {
        if ($name == "John Doe") {
            $prefix = "Mr.";
        } elseif ($name == "Jane Doe") {
            $prefix = "Mrs.";
        } else {
            $prefix = "";
        }
        return "{$prefix} {$name}";
    }
}
?>
```

Interface in PHP

```
<?php
interface Animal {
    public function makeSound();
}

class Cat implements Animal {
    public function makeSound() {
        echo " Meow ";
    }
}

class Dog implements Animal {
    public function makeSound() {
        echo " Bark ";
    }
}
?>
```

JavaScript & OOP

- ▶ JavaScript is not a class-based object-oriented language
- ▶ But it still has ways of using object-oriented programming (OOP)
- ▶ From ECMAScript 2015, class keyword was first introduced, as a “syntactic sugar” over the existing prototyping technique

Creating objects in JavaScript

- ▶ There are different ways to create new objects:
 - Create a single object, using an object literal
 - Create a single object, with the keyword `new`
 - Define an object constructor, and then create objects of the constructed type
 - Create an object using `Object.create()`

Creating object examples (1)

```
let person1 = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 50,  
  eyeColor: "blue"  
};
```

```
let person2 = {};  
person2.firstName = "John";  
person2.lastName = "Doe";  
person2.age = 50;  
person2.eyeColor = "blue";
```

Creating object examples (2)

```
let person3 = new Object();  
person3.firstName = "John";  
person3.lastName = "Doe";  
person3.age = 50;  
person3.eyeColor = "blue";
```


Accessing JavaScript properties

<code>objectName.property</code>	<code>// person.age</code>
<code>objectName["<i>property</i>"]</code>	<code>// person["age"]</code>
<code>objectName[expression]</code>	<code>// x = "age"; person[x]</code>

Creating JavaScript methods

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  id: 5566,  
  fullName: function() {  
    return this.firstName + " " + this.lastName;  
  }  
};  
  
console.log(person.fullName());
```

JavaScript Getter

// Create an object:

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  language: "en",  
  
  get lang() {  
    return this.language;  
  }  
};
```

// print using a getter:

```
console.log( person.lang );
```

JavaScript Setter

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  language: "",  
  
  set lang(lang) {  
    this.language = lang;  
  }  
};
```

```
// Set an object property using a setter:  
person.lang = "en";
```

JavaScript object constructors

```
function Person(first, last, age, eye) {  
  this.firstName = first;  
  this.lastName = last;  
  this.age = age;  
  this.eyeColor = eye;  
}  
  
let myFather = new Person("John", "Doe", 50, "blue");  
let myMother = new Person("Sally", "Rally", 48, "green");
```

JavaScript object prototypes

- All JavaScript objects inherit properties and methods from a prototype
- In the previous example we learned how to use an object constructor
- Please note that you can **not** add a new property to an existing object constructor
- To add a new property to a constructor, you **must add** it to **the constructor function**

Adding Properties and Methods to Objects

- ▶ Sometimes you want to add new properties (or methods) to an object constructor
- ▶ Sometimes you want to add new properties (or methods) to all existing objects of a given type

Using the prototype property (1)

```
function Person(first, last, age, eyecolor) {  
  this.firstName = first;  
  this.lastName = last;  
  this.age = age;  
  this.eyeColor = eyecolor;  
}
```

```
Person.prototype.nationality = "English";
```


Using the prototype property (1)

```
function Person(first, last, age, eyecolor) {  
  this.firstName = first;  
  this.lastName = last;  
  this.age = age;  
  this.eyeColor = eyecolor;  
}
```

```
Person.prototype.name = function() {  
  return this.firstName + " " + this.lastName;  
};
```

JavaScript keyword “class”

- ▶ ECMAScript 2015, also known as ES6, introduced JavaScript Classes
- ▶ JavaScript Classes are templates for JavaScript Objects
- ▶ You can use the keyword **class** to create a class and add a method named **constructor()**

Keyword "class" example (1)

```
class Car {  
    constructor(name, year) {  
        this.name = name;  
        this.year = year;  
    }  
}  
  
let myCar1 = new Car("Ford", 2014);  
let myCar2 = new Car("Audi", 2019);
```

Keyword "class" example (2)

```
class Car {  
  constructor(name, year) {  
    this.name = name;  
    this.year = year;  
  }  
  age() {  
    let date = new Date();  
    return date.getFullYear() - this.year;  
  }  
}  
  
let myCar = new Car("Ford", 2014);  
console.log("My car is " + myCar.age() + " years old.");
```

Getters and Setters

```
class Car {  
    constructor(brand) {  
        this.carname = brand;  
    }  
  
    get cnam() {  
        return this.carname;  
    }  
  
    set cnam(x) {  
        this.carname = x;  
    }  
}
```

JavaScript static methods

```
class Car {  
  constructor(name) {  
    this.name = name;  
  }  
  
  static hello() {  
    return "Hello!!";  
  }  
}
```

```
// You can call 'hello()' on the Car Class:  
console.log( Car.hello() );
```

JavaScript Class Inheritance

```
class Car {  
  constructor(brand) {  
    this.carname = brand;  
  }  
  present() {  
    return 'I have a ' + this.carname;  
  }  
}
```

```
class Model extends Car {  
  constructor(brand, mod) {  
    super(brand);  
    this.model = mod;  
  }  
  show() {  
    return this.present() + ', it is a ' + this.model;  
  }  
}
```

References

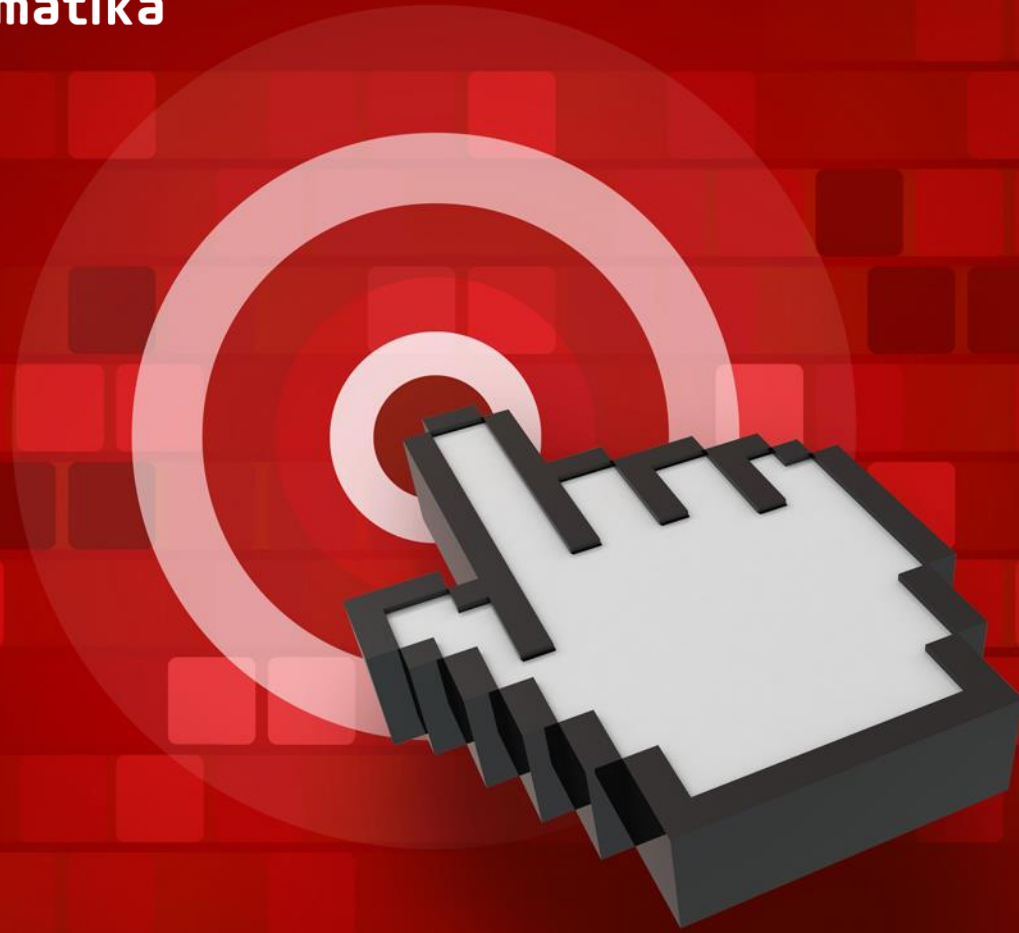
- <https://www.w3schools.com/php/>
- <https://www.w3schools.com/js/>

Question?





Fakultas Informatika
School of Computing
Telkom University



THANK YOU